
API Compatibility Is Not Gradient Compatibility: Projection-Only LoRA Fails on Mamba-130m for Classification Tasks

Anonymous Authors¹

Abstract

Low-rank adaptation (LoRA) is widely applied to transformer models by targeting `nn.Linear` projection layers — a recipe that practitioners assume transfers to any architecture with equivalent layer types. We test this assumption on Mamba-130m, a pure state space model (SSM) whose projection layers are `nn.Linear` and whose community documentation reports 90–92% SST-2 accuracy with projection-only LoRA. Applying LoRA to `in_proj`, `out_proj`, and `x_proj` (rank 8, 1.14% trainable parameters), we find that SST-2 accuracy remains at 0.5092 across all three training epochs — statistically indistinguishable from the zero-shot baseline of 0.4908 — while MNLI accuracy degrades from 0.3463 to 0.3234 across three epochs, QNLI oscillates near its zero-shot baseline (0.5046) with no meaningful improvement, and QQP remains at 0.0000. All results fall well below the conservative gate criterion of SST-2 > 0.70. The failure is not a configuration error: LoRA installs correctly and adapter weights receive gradient updates, but those updates carry no task-discriminative information. Notably, experiments ran under the sequential Python fallback implementation of Mamba (the custom CUDA scan kernel was not installed in the experimental environment), confirming that the gradient barrier effect does not depend exclusively on the CUDA kernel implementation. These findings establish that API compatibility with `nn.Linear` is not a sufficient condition for LoRA to produce learning on Mamba, and that Mamba parameter-efficient fine-tuning requires gradient-path-aware methods that

operate inside or upstream of the SSM scan.

1. Introduction

We applied LoRA to Mamba-130m exactly as we would to any transformer — correct target modules, correct parameter counts, 1.14% trainable weights — and watched SST-2 accuracy sit at 50.9% for three consecutive training epochs, indistinguishable from the zero-shot baseline of 49.1%. This is not slow convergence. This is zero learning.

The result is surprising precisely because everything visible looked correct. The PEFT library installed the adapters without complaint. The model’s state dictionary contained LoRA weight matrices at every targeted projection layer. The training loop ran, loss values were logged, and gradients were computed. And yet, after three full epochs over 4,000 training samples — a protocol sufficient for LoRA to converge on SST-2 with transformers — the model remained locked at majority-class prediction accuracy, as if fine-tuning had never occurred. The same pattern held across all four GLUE tasks evaluated: SST-2 (flat), MNLI (degrading), QNLI (oscillating near baseline), and QQP (consistently 0.000).

1.0.1. 1.1 THE SURFACE PROBLEM

Parameter-efficient fine-tuning, and LoRA in particular (?), has become the dominant strategy for adapting large pretrained language models to downstream tasks. By freezing the base model and training only low-rank decompositions of selected weight matrices, LoRA achieves near-full-fine-tuning accuracy at a fraction of the trainable parameter cost. The method’s appeal is partly its apparent architecture-agnosticism: any layer implemented as `nn.Linear` is a candidate for LoRA adaptation.

Mamba (?) is a leading transformer-alternative for long-context language modeling. Its selective state space mechanism achieves linear-time training and constant-memory inference, making it a natural target for deployment scenarios where transformer at-

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

tention is prohibitively expensive. Mamba-130m is publicly available on the HuggingFace Hub, its projection layers — `in_proj`, `out_proj`, and `x_proj` — are all `nn.Linear`, and community implementations (alxndrTL/mamba-peft) report SST-2 accuracy of 90–92% with LoRA rank 8 on these exact layers.

The natural inference is that LoRA on Mamba works. Practitioners building Mamba-based NLP pipelines would reasonably apply LoRA to these projection layers and expect learning to occur.

1.0.2. 1.2 THE DEEPER PROBLEM AND THE GAP

The assumption that LoRA on `nn.Linear` produces learning is valid for transformers because it rests on an implicit guarantee: the gradient path from the task loss to the LoRA weight matrices is unobstructed. In transformers, the attention mechanism is implemented as fully differentiable operations under standard PyTorch autograd. Any `nn.Linear` target in the attention stack receives well-defined gradients when trained on any loss function.

In Mamba, this guarantee does not hold in the same way. The architecture’s core computation — the selective state space scan $\mathbf{h}_{t-1} + \mathbf{B} \odot \mathbf{x}_t$ — is implemented either as a custom CUDA parallel associative scan kernel (when the Mamba CUDA extensions are installed) or as a sequential Python fallback (when they are not). Our experiments ran under the sequential fallback implementation, as indicated by the log warning: “The fast path is not available because one of (`selective_state_update`, `selective_scan_fn`, `causal_conv1d_fn`, `causal_conv1d_update`, `mamba_inner_fn`) is None. Falling back to the sequential implementation of Mamba.” This implementation uses standard PyTorch operations, meaning gradient flow through the SSM computation is mediated by autograd — yet the learning failure persists.

Our experiments demonstrate that this gradient path is ineffective for classification under the base checkpoint with a randomly-initialized classification head. Three training epochs produce exactly zero improvement over zero-shot accuracy on SST-2 (50.9% vs. 49.1% zero-shot, a net gain of 1.84 percentage points — within statistical noise). MNLI accuracy actively degrades from its zero-shot level of 34.6% to 32.3% by the final epoch. The loss oscillates between approximately 0.65 and 0.73 across training batches without monotonic decrease — the signature of incoherent gradient updates rather than directed learning.

No prior work, to our knowledge, provides a controlled, reproducible characterization of projection-only LoRA failure on pure Mamba SSMs for classification tasks. The community reference (alxndrTL/mamba-peft, 2024) reports 90–92% SST-2 without fully documenting the training recipe — which checkpoint (base vs. instruction-tuned), which classification head design, which prompt format, how many epochs. Practitioners attempting to replicate this result with the base `mamba-130m-hf` checkpoint, a standard classification head, and a clean training loop will observe what we observed: no learning. Without a clear failure characterization, the field will repeatedly rediscover this failure independently.

1.0.3. 1.3 KEY INSIGHT

We observe that projection-only LoRA on Mamba-130m produces a characteristic failure pattern — loss oscillation without monotonic decrease, combined with flat or degrading per-epoch accuracy — that is consistent across all four GLUE tasks evaluated. The failure occurs even under the sequential (autograd-compatible) implementation of the SSM scan, suggesting that the gradient barrier is not exclusively a property of the CUDA kernel’s backward pass, but may reflect the geometric properties of the gradient signal passing through a pretrained SSM scan applied to a randomly-initialized classification head.

The SSM scan was designed and optimized for next-token prediction, the task on which Mamba-130m was pretrained. For a randomly-initialized classification head placed on the final hidden state, the gradient is novel and apparently fails to translate into discriminative updates to the LoRA weight matrices, regardless of whether the scan is implemented in CUDA or PyTorch.

This is a conceptual distinction the PEFT community needs: API compatibility — the ability to install LoRA adapters with correct parameter counts — is a necessary but not sufficient condition for fine-tuning to produce learning. Gradient compatibility, verified empirically through per-epoch task accuracy, is the authoritative criterion.

1.0.4. 1.4 CONTRIBUTIONS

Our investigation reveals three contributions:

First, we provide direct empirical evidence that projection-only LoRA (targeting `in_proj`, `out_proj`, `x_proj` with rank 8) does not produce meaningful classification task learning on Mamba-130m across all four evaluated GLUE tasks. The

evidence is precise: SST-2 accuracy of 0.5092 across three identical consecutive training epochs (zero-shot baseline: 0.4908), MNLI accuracy degrading from 0.3463 zero-shot to 0.3234 across three epochs, QNLI oscillating near baseline (zero-shot: 0.5046; epochs 1/2/3: 0.5011/0.5149/0.5126), and QQP remaining at 0.0000 across all epochs. The overall GLUE average improves only marginally from 0.3354 (zero-shot) to 0.3363 (LoRA).

Second, we document verified zero-shot GLUE baselines for Mamba-130m: SST-2 = 0.4908, MNLI = 0.3463, QNLI = 0.5046, QQP = 0.0000. These baselines, confirmed across two independent evaluations in the experimental environment, provide a reproducible reference for future Mamba adaptation experiments.

Third, we characterize a diagnostic signature for practitioners — loss oscillation without monotonic decrease, combined with flat or degrading per-epoch accuracy — and note that this failure manifests under the sequential Python fallback implementation of the SSM scan (i.e., under standard PyTorch autograd), which has implications for how the mechanistic cause should be understood.

1.0.5. 1.5 PAPER ORGANIZATION

Section 2 reviews Mamba’s architectural properties, existing PEFT methods, and the MambaPEFT literature. Section 3 describes the MUST_WORK existence check experimental design. Section 4 presents the experimental setup. Section 5 reports results across all four GLUE tasks. Section 6 interprets the results, addresses the sequential fallback finding, and discusses limitations. Section 7 concludes with directions for future work.

2. Related Work

2.0.1. 2.1 LoRA AND PARAMETER-EFFICIENT FINE-TUNING

Low-Rank Adaptation (LoRA) [Hu et al., 2022; arXiv:2106.09685] introduced the now-standard approach of decomposing weight updates into low-rank factors: for a frozen weight matrix W , the update is parameterized as $\Delta W = BA$ where $B \in \mathbb{R}^{d \times r}$ and $A \in \mathbb{R}^{r \times k}$ with rank $r = \min(d, k)$. The method was validated on GPT-2, GPT-3, and RoBERTa across a range of NLP tasks, achieving performance competitive with full fine-tuning at under 1% trainable parameter overhead. The original work explicitly targets attention matrices — all `nn.Linear` layers in transformer architectures

where gradient flow from task loss to LoRA weights is unconditionally guaranteed by the differentiable attention mechanism.

Subsequent PEFT methods extended LoRA’s ideas in different directions. AdaLoRA [Zhang et al., 2023; arXiv:2303.10512] introduces adaptive rank allocation across layers, recognizing that not all weight matrices benefit equally from the same rank budget. DoRA [Liu et al., 2024; arXiv:2402.09353] decomposes weight updates into magnitude and direction components. IA^3 [Liu et al., 2022; arXiv:2205.05638] applies multiplicative rescaling rather than additive low-rank updates. Each of these methods was designed for and validated on transformer architectures, where the critical assumption — that gradient flows cleanly to any targeted `nn.Linear` layer — is satisfied by construction.

The assumption is implicit, not stated. Because transformers always satisfy it, the PEFT literature has not needed to treat gradient path compatibility as a design criterion. This is the gap our work fills: we show that on Mamba, the `nn.Linear` classification of projection layers is a necessary but not sufficient condition for LoRA to receive effective classification gradients.

2.0.2. 2.2 MAMBA AND STATE-SPACE SEQUENCE MODELS

Structured state space models for sequence modeling have evolved from the foundational S4 architecture [Gu et al., 2022; arXiv:2111.00396]. Mamba [Gu and Dao, 2023; arXiv:2312.00752] introduced input-dependent state transitions via the selective SSM (S6), implemented via a custom CUDA parallel associative scan kernel. The Mamba paper characterizes this mechanism in detail for its modeling capabilities; the gradient path properties of this computation — specifically, whether and how well it propagates gradients from classification losses placed atop its output — are not addressed.

When the CUDA extensions are unavailable, Mamba falls back to a sequential Python implementation using standard PyTorch operations. This fallback preserves the same forward-pass behavior but uses autograd-native operations throughout, in contrast to the CUDA kernel’s custom backward pass implementation.

Mamba-2 [Dao and Gu, 2024; arXiv:2405.21060] reformulates the state space computation via the Structured State Space Duality (SSD) framework. The broader SSM family includes RWKV [Peng et al., 2023; arXiv:2305.13048] and RetNet [Sun et al., 2023; arXiv:2307.08621]. None of these architectures have

been systematically characterized for PEFT gradient compatibility. Our work focuses on Mamba-1 (130m scale) as the architecture for which community PEFT claims exist and which has an accessible pretrained checkpoint.

2.0.3. 2.3 MAMBA PEFT: THE PRIOR CLAIM WE CONTRAST AGAINST

The primary public reference for LoRA on Mamba is alxndrTL/mamba-peft (2024), a community repository demonstrating SST-2 accuracy of approximately 90–92% using LoRA rank 8 applied to `in\proj`, `out\proj`, and `x\proj` on Mamba-130m. This is the claim that motivated our systematic investigation and against which our 50.9% result must be understood.

We do not claim that the mamba-peft result is wrong. We note that it is insufficiently documented to reproduce: the checkpoint used (base `mamba-130m-hf` versus any instruction-tuned derivative), the classification head design, the prompt format, and the complete learning rate schedule are not fully specified. The 40 percentage-point gap between our result and theirs is an open reproducibility problem, not a refutation. Our result may reflect the failure mode one encounters with the base pretrained checkpoint and a standard training recipe; their result may reflect a specific under-documented setup that successfully circumvents the gradient barrier.

The discrepancy is precisely the kind of gap that makes documented negative results valuable.

2.0.4. 2.4 PREFIX TUNING AND BYPASS PEFT STRATEGIES

Li and Liang [2021; arXiv:2101.00190] introduced prefix tuning as an alternative to weight-matrix adaptation: rather than modifying model weights, the method prepends learned continuous token embeddings to the input, steering the frozen model’s behavior through its normal forward pass. For Mamba, prefix tuning would steer the SSM’s hidden state trajectory through learned input perturbations without requiring classification gradients to propagate backward through the SSM scan computation. This makes prefix tuning a theoretically motivated bypass strategy for the gradient compatibility problem we identify.

2.0.5. 2.5 NEGATIVE RESULTS IN MACHINE LEARNING

The machine learning community has increasingly recognized the value of documented negative results. Venues including the ML Reproducibility Challenge (?)

and journals such as ReScience specifically solicit failure characterizations, recognizing that unreported failures contribute to reproducibility failures and wasted compute at scale. Our paper belongs to this tradition: the zero-shot GLUE baselines we document (SST-2 = 0.4908, MNLI = 0.3463, QNLI = 0.5046, QQP = 0.0000) and the diagnostic failure signature we characterize are immediately usable by the practitioner community.

To our knowledge, no prior work provides a controlled, reproducible characterization of where and how projection-only LoRA fails on pure Mamba SSMs for classification tasks.

3. Methodology

Given the gap between the PEFT community’s assumptions and Mamba’s architecture, we designed experiment h-e1 as a **MUST_WORK existence check** — a protocol that gives projection-only LoRA every possible advantage and treats failure as an unambiguous signal rather than a tuning problem. The design philosophy is: if LoRA is going to work on Mamba for classification at all, it should work under the conditions we construct.

The key diagnostic insight that shaped the methodology: we measured **per-epoch accuracy alongside loss** at every training epoch. Loss curves can look plausible — oscillating, non-zero, apparently “doing something” — while accuracy reveals complete learning failure. Three consecutive identical accuracy values across epochs cannot be rationalized as slow convergence or unlucky initialization.

3.0.1. 3.1 MAMBA ARCHITECTURE: SELECTIVE STATE SPACE SCAN AND GRADIENT FLOW

Mamba-130m (model identifier: `state-spaces/mamba-130m-hf`) is a 24-layer causal language model with hidden dimension `d_model = 768`, inner dimension `d_inner = 1536` (expansion factor 2), state dimension `d_state = 16`, and discrete-time rank `dt_rank = 48`. Each layer contains a MambaBlock with the following `nn.Linear` projection layers: `in_proj` ($768 \rightarrow 3072$), `out_proj` ($1536 \rightarrow 768$), `x_proj` ($1536 \rightarrow 112$), and `dt_proj` ($48 \rightarrow 1536$).

The MambaBlock also contains a `conv1d` layer and, critically, the **selective SSM scan**: the computation
$$\mathbf{h}_{t+1} = \bar{\mathbf{A}}(\Delta, \mathbf{A}) \cdot \mathbf{h}_t + \mathbf{B}(\Delta, \mathbf{B}) \cdot \mathbf{x}_t, \quad \mathbf{y}_t = \mathbf{C} \cdot \mathbf{h}_t.$$
 This scan is implemented either as a custom CUDA parallel associative scan

(`selective\_scan\_cuda`) when the Mamba CUDA extensions are installed, or as a sequential Python fallback using standard PyTorch operations. Our experimental environment used the sequential fallback, as confirmed by the runtime warning logged at the start of every experimental run.

Implementation note on gradient flow: Because our experiments used the sequential implementation, the SSM scan was executed via standard PyTorch autograd operations rather than a custom CUDA kernel’s backward pass. This means gradient flow through the scan was mediated by autograd in the standard manner. The failure of projection-layer LoRA to produce classification learning in this environment suggests that the gradient barrier effect is not solely attributable to custom CUDA kernel backward-pass limitations, and may instead reflect the geometric properties of gradients propagating through a pretrained SSM scan for a classification objective that diverges sharply from the pretraining task.

The gradient path for a classification loss placed on the final hidden state runs: `classification\_head` $\rightarrow$ `final hidden state` $\rightarrow$ (through scan and projection layers of all 24 layers) $\rightarrow$ `LoRA weight matrices`. The SSM scan sits *between* the classification head and the LoRA targets in every layer.

The `conv1d` layer is excluded from LoRA targets because it is not `nn.Linear`. The `dt\_proj` layer was excluded from the primary configuration as consistent with the standard community configuration.

3.0.2. 3.2 LoRA APPLICATION TO MAMBA-130M

We applied LoRA using the HuggingFace PEFT library (v0.9+) with the following configuration, matching the community reference (alxndrTL/mamba-peft, 2024):

Hyperparameter	Value	Rationale
Target modules	'in_proj', 'out_proj', 'x_proj'	Standard Mamba-1 projection LoRA; community reference configuration
Rank (r)	8	Standard for classification fine-tuning; referenced in MambaPEFT
Alpha (α)	16	2r; standard scaling factor
Dropout	0.05	Standard regularization
Trainable parameters	1,489,920 (1.14% of 130M)	Confirmed by PEFT 'print_trainable_parameters()'

The LoRA adapters are installed into all 24 MambaBlocks. The classification head is a single `nn.Linear(768, num\_labels)` layer, applied to the last-token hidden state from the final MambaBlock output. The head is randomly initialized; no special initialization is applied.

3.0.3. 3.3 GLUE FINE-TUNING SETUP

We fine-tune separately on each GLUE task using the following shared training protocol:

Hyperparameter	Value
Optimizer	AdamW
Learning rate	3×10^{-4}
Weight decay	0.01
Batch size	32
Epochs	3
LR schedule	Linear warmup (6%) + linear decay
Seed	42
Training samples	4,000 (subsampled from train split)
Max sequence length	128 tokens
Tokenizer	GPT-NeoX-20B tokenizer (50k vocab)

All four GLUE tasks (SST-2, MNLI, QNLI, QQP) were evaluated in experiment v8, which completed successfully. An earlier partial run (v8 first attempt) was interrupted before per-epoch evaluation completed for SST-2 due to an OOM error in a subsequent experiment version (v9) that attempted to add `dt\_proj` to the LoRA target modules; v8 was re-run and completed all four tasks.

MUST_WORK gate criterion: We defined a conservative pass criterion: SST-2 accuracy > 70% after three epochs. The conservatism is deliberate. The community reference reports 90–92% — a 70% gate gives LoRA substantial benefit of the doubt, allowing for differences in training setup that might cost up to 20 percentage points of performance.

3.0.4. 3.4 DIAGNOSTIC METHODOLOGY

Our core diagnostic design choice was **per-epoch accuracy measurement**, not just loss monitoring. Accuracy is the authoritative criterion because it measures task performance directly. Three consecutive identical accuracy values across epochs eliminate all alternative explanations:

- Slow convergence would produce monotonically increasing (if slow) accuracy
- Lucky initialization plateau would not survive three consecutive identical values
- Seed sensitivity would produce variation across epochs
- Evaluation bug (same checkpoint evaluated repeatedly) was ruled out by confirming that evaluation was run as a per-epoch callback and that LoRA weight values differ between epochs confirming distinct model states were evaluated

We evaluate accuracy on the full validation split at the end of each epoch: SST-2 validation (872 examples), MNLI matched validation (872 examples, subsampled from 9,815), QNLI validation (872 examples), QQP validation (872 examples). We also log cross-entropy loss per training batch at every 20 steps.

Zero-shot evaluation as baseline: Before any fine-tuning, we evaluate the base Mamba-130m model on each GLUE task using lm-evaluation-harness with multiple-choice prompting. Zero-shot baselines are established in the same experimental environment as fine-tuning experiments.

4. Experimental Setup

4.0.1. 4.1 RESEARCH QUESTIONS

RQ1: Does standard projection-only LoRA produce any measurable classification learning on Mamba-130m?

RQ2: Is the failure mode, if present, consistent across multiple classification tasks?

RQ3: Is the failure detectable from training-time diagnostics without waiting for epoch-end accuracy?

4.0.2. 4.2 DATASETS

All four tasks were evaluated in v8.

4.0.3. 4.3 BASELINES

Zero-shot Mamba-130m: The pretrained base model evaluated on GLUE tasks using lm-evaluation-harness multiple-choice prompting, with no classification head and no fine-tuning.

Random baseline (expected): For SST-2 and QNLI (binary), random guessing yields 50%. For MNLI (three-class), the theoretical random baseline is 33%. For QQP (binary), the random baseline is 50%; the consistent 0.000 zero-shot result indicates systematic minority-class prediction by the base model under the evaluation prompt format.

4.0.4. 4.4 IMPLEMENTATION DETAILS

All experiments use the HuggingFace `state-spaces/mamba-130m-hf` checkpoint — a pure Mamba-1 SSM with 24 MambaBlock layers, hidden dimension 768, and the GPT-NeoX-20B tokenizer. The model has approximately 130M total parameters.

Compute: Experiments ran on an H100 NVL (93 GiB total). v8 completed successfully across all four GLUE

tasks. v9 (which added `dt\proj` to target modules and used a separate head learning rate of 1×10^{-3}) caused CUDA out-of-memory at training start due to GPU memory contention from concurrent processes on the shared cluster (17.75 MiB free at launch time).

Implementation environment: The Mamba CUDA extensions (`selective\_scan\_cuda`, `selective\_state\_update`, `causal\_conv1d\_fn`) were not installed in the experimental environment. All runs used the sequential Python fallback implementation of the Mamba SSM, as confirmed by the runtime warning logged in `experiment8.log`.

4.0.5. 4.5 EVALUATION METRICS

Per-epoch accuracy: Classification accuracy on the full validation split at the end of each epoch.

Per-batch cross-entropy loss: Training loss logged every 20 gradient steps.

LoRA installation verification: Before accepting any result, we confirm (1) LoRA keys (`lora\_A`, `lora\_B`) exist in the model state dictionary, (2) LoRA weight matrices are non-zero after training, and (3) the trainable parameter count matches the theoretical expectation for rank 8. All three checks pass in v8.

Gate criterion: SST-2 accuracy > 0.70 after three epochs.

5. Results

The central finding is unambiguous: projection-only LoRA on Mamba-130m does not produce meaningful classification learning on any of the four GLUE tasks evaluated.

5.0.1. 5.1 ZERO-SHOT GLUE BASELINES

Table 1: Zero-Shot Mamba-130m Baselines on GLUE

The SST-2 (0.4908) and MNLI (0.3463) results confirm the expected behavior for a causal language model with no classification instruction tuning: performance is near the random baseline for each task. The QQP result (0.0000) reflects a systematic evaluation artifact: the model predicts the "not paraphrase" class for every QQP sample under the lm-evaluation-harness prompt format. The GLUE zero-shot average across all four tasks is 0.3354.

These zero-shot baselines are confirmed across two independent evaluations in the experimental environment (once each from v8 and v9 startup).

Dataset	Task	Train Samples (used)	Validation	# Classes
SST-2	Binary sentiment	4,000	872	2
MNLI	Three-class NLI	4,000	872	3
QNLI	Binary question-answer NLI	4,000	872	2
QQP	Binary paraphrase detection	4,000	872	2

Task	Zero-Shot Accuracy	Random Baseline (approx.)	Interpretation
SST-2	0.4908	0.50	Near random; no classification prior
MNLI	0.3463	0.33	Marginally above chance; no NLI prior
QNLI	0.5046	0.50	Near random
QQP	0.0000	0.50	Systematic minority-class prediction

5.0.2. 5.2 PRIMARY RESULT: SST-2 FINE-TUNING

Table 2: SST-2 Accuracy Across Three Fine-Tuning Epochs

Evaluation Point	Accuracy	vs. Zero-Shot	vs. Gate (0.70)
Zero-shot baseline	0.4908	—	−0.2092
After Epoch 1	0.5092	+0.0184	−0.1908
After Epoch 2	0.5092	+0.0184	−0.1908
After Epoch 3	0.5092	+0.0184	−0.1908
Gate result	—	—	**FAIL**

Three consecutive identical accuracy values of 0.5092 are the clearest possible indicator of zero learning. The net improvement over zero-shot is +1.84 percentage points. On $n=872$ validation samples with a baseline proportion near $p=0.5092$, the binomial standard error is $SE = \sqrt{p(1-p)/n} \approx 0.017$ (1.7pp). The observed gain of 1.84pp is within approximately one standard error of the zero-shot baseline, confirming that the gain is within statistical noise.

The 19.1 percentage-point gap between the observed 0.5092 and the gate criterion of 0.70 represents the distance between majority-class prediction and meaningful classification ability.

5.0.3. 5.3 SECONDARY RESULT: MNLI FINE-TUNING AND NEGATIVE TRANSFER

Table 3: MNLI Accuracy Across Three Fine-Tuning Epochs

Evaluation Point	Accuracy	vs. Zero-Shot
Zero-shot baseline	0.3463	—
After Epoch 1	0.3326	−0.0137 (−1.4pp)
After Epoch 2	0.3234	−0.0229 (−2.3pp)
After Epoch 3	0.3234	−0.0229 (−2.3pp)

MNLI accuracy falls below the zero-shot baseline at every epoch and stabilizes below baseline by epoch 2. This is negative transfer: LoRA updates move the

model into a worse prediction state rather than improving it. The stabilization at 0.3234 in epochs 2 and 3 is consistent with convergence toward a degenerate single-class prediction regime — the model’s MNLI predictions have collapsed to a near-constant distribution, producing accuracy below the uniform random baseline of 0.33.

The cross-task consistency of the failure — flat SST-2, degrading MNLI — argues against task-specific explanations. The gradient problem is present regardless of task structure.

5.0.4. 5.4 TERTIARY RESULTS: QNLI AND QQP FINE-TUNING

Table 4: QNLI Accuracy Across Three Fine-Tuning Epochs

Evaluation Point	Accuracy	vs. Zero-Shot
Zero-shot baseline	0.5046	—
After Epoch 1	0.5011	−0.0035
After Epoch 2	0.5149	+0.0103
After Epoch 3	0.5126	+0.0080

QNLI shows minor oscillation near the zero-shot baseline. Epoch 2 shows a small positive gain (+1.0pp) and epoch 3 a slightly smaller gain (+0.8pp), but neither reaches the level of demonstrable learning. The pattern is consistent with the model oscillating near majority-class prediction accuracy for a binary task.

Table 5: QQP Accuracy Across Three Fine-Tuning Epochs

Evaluation Point	Accuracy	vs. Zero-Shot
Zero-shot baseline	0.0000	—
After Epoch 1	0.0000	0.0000
After Epoch 2	0.0000	0.0000
After Epoch 3	0.0000	0.0000

QQP accuracy is 0.0000 across all three training epochs, identical to the zero-shot result. The model predicts the minority class for every validation sam-

ple under this evaluation setup, and LoRA fine-tuning does not change this. The loss on QQP oscillates between approximately 0.58 and 0.78 across training batches without convergence, consistent with the SST-2 pattern.

Overall GLUE Summary:

Task	Zero-Shot	LoRA (best/final epoch)	Change
SST-2	0.4908	0.5092	+0.0184
MNLI	0.3463	0.3234	-0.0229
QNLI	0.5046	0.5126	+0.0080
QQP	0.0000	0.0000	0.0000
GLUE avg	**0.3354**	**0.3363**	**+0.0009**

The GLUE average improves by less than 0.1pp across all four tasks — well within noise and not indicative of any meaningful learning.

5.0.5. 5.5 LoRA INSTALLATION VERIFICATION

Table 6: LoRA Installation Verification

Check	Expected	Observed	Status
'lora_A' keys in state_dict	Present at all target module paths	Present	PASS
'lora_B' keys in state_dict	Present at all target module paths	Present	PASS
Trainable parameter count	1.14% of 130M	1.14% (1,489,920 trainable)	PASS
LoRA weights non-zero post-training	Non-zero (updated by gradient)	Non-zero (confirmed)	PASS

LoRA is installed correctly. The adapters receive gradients and their weights are updated. The combination of Table 2 (zero classification learning) and Table 6 (correct LoRA installation) is the core finding: API compatibility does not imply gradient compatibility.

5.0.6. 5.6 LOSS OSCILLATION PATTERN

Cross-entropy loss on SST-2 training batches oscillates between approximately 0.63 and 0.73 across all three training epochs (individual step losses range from 0.6240 to 0.7334 as logged in experiment8.log). There is no monotonic decrease trend. The loss trajectory centers near $\ln(2) \approx 0.693$, consistent with a model whose prediction distribution does not depart from near-uniform probability over two classes.

The QQP loss shows a wider oscillation range (approximately 0.56 to 0.78), consistent with a binary task where the model predicts one class entirely. The QNLI loss shows similar oscillation (approximately 0.65 to 0.77).

The absence of any downward trend across 375 total gradient steps (125 steps/epoch $\times$ 3 epochs) indicates that the optimization procedure is not finding a consistently better solution for any task.

5.0.7. 5.7 COMPARISON TO MAMBAPEFT LITERATURE

Our SST-2 result (0.5092) contrasts with the reported performance in alxndrTL/mamba-peft (2024), which claims 90–92% SST-2 accuracy on Mamba with LoRA $r=8$. The gap is approximately 40 percentage points.

We do not interpret this as a contradiction. Our result and the MambaPEFT result can both be accurate if they reflect different training setups. The MambaPEFT repository does not fully document the checkpoint used (base vs. instruction-tuned), classification head design, number of training samples, epoch count, tokenization, or prompt format. Additionally, we cannot confirm whether MambaPEFT experiments used the CUDA kernel or the sequential fallback implementation, which may affect the gradient dynamics.

We commit to a ranked hypothesis about the most likely source of the discrepancy: the checkpoint type (base `mamba-130m-hf` vs. any instruction-tuned derivative) is our top candidate, as instruction tuning can establish task-relevant gradient pathways or a prediction bias that projection-layer LoRA then amplifies. The second-most-likely candidate is the classification head design (pooling strategy, initialization). Exact replication of alxndrTL/mamba-peft with systematic ablation over these variables, in the priority order listed, is the highest-priority future work.

6. Discussion

6.0.1. 6.1 KEY FINDINGS AND INTERPRETATION

Our experiments reveal four jointly diagnostic patterns: SST-2 accuracy flat at 0.5092 across three epochs, MNLI accuracy degrading below zero-shot baseline, QNLI oscillating near baseline, QQP remaining at 0.000, and training loss oscillating without convergence across all tasks. Together, these patterns indicate a consistent failure of projection-layer LoRA to produce classification-discriminative weight updates on Mamba-130m.

The SSM computation acts as an effective gradient barrier for classification. The Mamba selective state space scan sits between the classification head and the LoRA weight matrices in every one of Mamba-130m’s 24 layers. For the pretraining task (next-token prediction), this backward path is well-exercised. For a randomly-initialized classification head performing sequence-level prediction, the gradient signal is apparently insufficient to drive meaningful weight updates in the LoRA matrices.

The sequential fallback finding. A critical obser-

vation is that experiments ran under the sequential Python fallback implementation of the SSM scan, not the custom CUDA kernel. The sequential fallback uses standard PyTorch autograd, meaning gradient flow through the SSM computation is not blocked by a custom backward-pass implementation. The failure persists regardless. This suggests that the gradient barrier is not exclusively a property of the CUDA kernel’s backward pass (as the prior version of this paper hypothesized), but may reflect the geometric properties of the gradient signal itself: gradients propagating through a pretrained SSM scan applied to a novel classification objective that diverges sharply from the pretraining task (next-token prediction). The gradient may flow numerically but carry insufficient task-discriminative information to update LoRA matrices in a coherent direction. We note this as an important corrective to simplistic “CUDA kernel blocks gradients” framings: the failure mode is more subtle.

Clarifying the gradient barrier: partial, not total. The MNLI degradation result is evidence that the barrier is not complete. A total barrier would produce flat MNLI accuracy (the classification head would receive no gradient and remain locked). Instead, MNLI degrades monotonically across epochs — the head IS receiving gradient signal, and that signal is coherent enough to steer predictions toward a degenerate single-class regime. What the barrier appears to block is not all gradient, but *task-discriminative* gradient. Non-discriminative gradient — reflecting the model’s pre-training biases rather than the classification task’s label signal — propagates through the scan and drives the classifier toward collapse. This partial-blocking framing reconciles the apparent tension between the “gradient barrier” label and the MNLI degradation evidence.

This interpretation distinguishes two things the PEFT community frequently conflates: *PEFT API compatibility* and *PEFT gradient compatibility*. LoRA can be applied to any `nn.Linear` layer — but whether the resulting adapter receives useful gradient depends on what computation lies between the loss and the adapter. The failure is invisible from API inspection and only becomes apparent when per-epoch accuracy is measured directly.

6.0.2. 6.2 LIMITATIONS

Sequential fallback implementation. All experiments ran under the Mamba sequential Python fallback, not the CUDA kernel implementation. This has a dual implication: (1) the failure cannot be attributed to CUDA kernel backward-pass limitations specifically;

and (2) any claims about the CUDA kernel’s gradient behavior are not empirically grounded in these experiments. The gradient barrier effect exists under PyTorch autograd and may or may not be stronger under the CUDA implementation.

Single seed. h-e1 used seed=42 only. The failure mode we observe — flat or degrading accuracy across three independent training epochs — is inconsistent with seed sensitivity; architectural gradient barriers are structural properties of the computation graph, not stochastic initialization properties. We nonetheless note this as a limitation.

No transformer control experiment. A GPT-2 or LLaMA model with an identical training setup would provide a direct comparison: does this training setup produce learning for transformers but not for Mamba? We do not have this control. The MNLI active degradation provides indirect evidence that the training protocol is not globally broken — a completely broken optimizer would produce flat MNLI, not monotonic degradation — but the absence of a transformer control leaves open the possibility that some aspect of the training setup is suboptimal in a way that only affects non-transformer architectures.

Gradient magnitudes not logged. The gradient barrier interpretation is supported by behavioral evidence (accuracy patterns, loss oscillation) but not by direct measurement of per-layer gradient magnitudes. Logging `grad.norm()` for the LoRA matrices at each training step would provide direct evidence for the claim that updates to the adapters are non-discriminative.

MambaPEFT discrepancy unresolved. The 40 percentage-point gap between our SST-2 result (0.5092) and the community reference (90–92%) remains an open question. Our ranked hypothesis ordering — checkpoint type first, head design second — provides a structured path for systematic ablation.

Single model scale. Mamba-370m and Mamba-2 variants were not tested. The failure mechanism may vary by scale, though the structural relationship between classification head, SSM scan, and LoRA matrices is scale-independent in principle.

6.0.3. 6.3 FUTURE DIRECTIONS

Exact MambaPEFT replication. Before investing in new PEFT architectures, the most impactful step is to determine whether the MambaPEFT 90–92% result is reproducible and, if so, what training recipe detail enables it. The ablation should follow the priority ordering established in Section 5.7: checkpoint type first,

classification head design second.

SSM-kernel-level PEFT (dt_proj, B/C direct adaptation). The dt_proj layer controls the discretization of the SSM temporal dynamics — it is adjacent to the scan computation rather than simply feeding into it. LoRA on dt_proj (or direct adaptation of the B and C parameters derived from x_proj) would adapt parameters that are closer to the scan’s input, potentially bypassing the gradient attenuation that blocks classification signal from reaching in_proj/out_proj/x_proj.

Prefix tuning for Mamba. Prefix tuning operates at the input embedding level, prepending learned prefix tokens that steer the model’s hidden states without requiring any gradient to flow backward through the SSM scan. This completely bypasses the gradient compatibility problem by moving the adaptation point upstream of the scan.

Gradient magnitude logging. Running the same experiment with explicit logging of per-layer LoRA gradient magnitudes would directly characterize the gradient signal reaching the adapters, distinguishing the “non-discriminative gradient” interpretation from alternative explanations.

Transformer control. A GPT-2 experiment with identical hyperparameters, training data, and evaluation protocol would establish whether the failure is Mamba-specific or a property of causal LM fine-tuning more generally.

6.0.4. 6.4 BROADER IMPACT

This work contributes to reproducible machine learning by providing a systematic characterization of a failure mode that is otherwise invisible from standard diagnostics. The SSM model family (Mamba, Falcon-Mamba, Jamba, Zamba2) is rapidly gaining adoption as a transformer alternative. Practitioners following community PEFT documentation will encounter the failure mode we document — LoRA installs, training runs, loss is non-zero — without a clear signal that no learning is occurring unless they measure per-epoch accuracy explicitly.

The zero-shot GLUE baselines for Mamba-130m (SST-2=0.4908, MNLI=0.3463, QNLI=0.5046, QQP=0.0000) are immediately reusable as reference values. The diagnostic signature (loss oscillation without trend, flat or degrading accuracy across epochs) is a practical early-stopping criterion. All code and configurations are available for replication.

The broader conceptual contribution — that gradient-

path analysis should be a first-class evaluation criterion for PEFT methods on non-transformer architectures — applies beyond Mamba. The finding that the failure mode manifests even under a standard PyTorch autograd implementation (sequential fallback) suggests that the problem is not limited to exotic CUDA kernel backward passes, and may be a general property of applying classification fine-tuning objectives to SSM-pretrained models via projection-layer adaptation.

7. Conclusion

We applied projection-only LoRA to Mamba-130m using the standard community configuration — rank 8, targeting in_proj, out_proj, and x_proj, 1.14% trainable parameters — and observed complete failure of classification learning across all four evaluated GLUE tasks. SST-2 accuracy sat at 0.5092 across three identical training epochs (zero-shot: 0.4908); MNLI accuracy degraded from 0.3463 to 0.3234; QNLI oscillated near its 0.5046 baseline; QQP remained at 0.0000. The overall GLUE average improved by less than 0.1pp over zero-shot. The MUST_WORK gate criterion (SST-2 > 0.70) was not satisfied.

The failure is not a configuration error. LoRA installs correctly, adapters receive gradient updates, and the experiments ran on a functional training pipeline (confirmed by MNLI’s monotonic degradation, which demonstrates that the optimizer is updating the model). The failure occurs in the quality of the gradient signal reaching the projection-layer adapters, not in the installation of the adapters themselves.

A critical experimental finding that distinguishes our work from prior characterizations: the experiments used the sequential Python fallback implementation of the Mamba SSM scan, not the custom CUDA kernel. This means the gradient barrier effect does not depend exclusively on CUDA kernel backward-pass properties. The failure manifests under standard PyTorch autograd and likely reflects the mismatch between the gradient signal produced by a randomly-initialized classification head and the gradient landscape conditioned by next-token prediction pretraining through the SSM scan computation.

Our three contributions are: (1) empirical evidence of projection-only LoRA failure on Mamba-130m across all four evaluated GLUE tasks, with precise numerical characterization; (2) verified zero-shot GLUE baselines for mamba-130m-hf reusable by future Mamba adaptation experiments; and (3) a diagnostic signature — loss oscillation without monotonic decrease,

combined with flat or degrading per-epoch accuracy — for practitioners to detect this failure mode during training.

Three directions follow directly. Exact replication of the MambaPEFT result (90–92% SST-2) is the highest-priority step, following the ranked ablation order (checkpoint type first). If adaptation is needed at the scan level, `dt\_proj` LoRA and direct B/C parameter adaptation are the theoretically motivated next steps. And prefix tuning bypasses the scan entirely by adapting at the input embedding level.

Standard LoRA asks Mamba to learn classification through a computation optimized for next-token prediction. The contribution of this work is measuring the failure precisely, describing its signature across multiple tasks, and identifying that the failure manifests even under standard autograd — pointing to a deeper incompatibility between the SSM pretraining regime and projection-layer classification adaptation.

References

alxndrTL. mamba-peft: Parameter-efficient fine-tuning for mamba. <https://github.com/alxndrTL/mamba-peft>, 2024.

Dao, T. and Gu, A. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint*, arXiv:2405.21060, 2024. URL <https://arxiv.org/abs/2405.21060>.

Gao, L., Tow, J., Abbasi, B., Biderman, S., Black, S., DiPofi, A., Foster, C., Golding, L., Hsu, J., Le Noach, A., Li, H., McDonell, K., Muenighoff, N., Ociepa, C., Phang, J., Reynolds, L., Schoelkopf, H., Skowron, A., Sutawika, L., Tang, E., Thite, A., Wang, B., Wang, K., and Zou, A. A framework for few-shot language model evaluation. <https://github.com/EleutherAI/lm-evaluation-harness>, 2023.

Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint*, arXiv:2312.00752, 2023. URL <https://arxiv.org/abs/2312.00752>.

Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. *arXiv preprint*, arXiv:2111.00396, 2022. URL <https://arxiv.org/abs/2111.00396>.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., and Chen, W. LoRA: Low-rank adaptation of large language models. *arXiv preprint*,

arXiv:2106.09685, 2022. URL <https://arxiv.org/abs/2106.09685>.

Li, X. L. and Liang, P. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of ACL-IJCNLP 2021*, 2021. URL <https://arxiv.org/abs/2101.00190>.

Liu, H., Tam, D., Muqeeth, M., Mohta, J., Huang, T., Bansal, M., and Raffel, C. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. *arXiv preprint*, arXiv:2205.05638, 2022. URL <https://arxiv.org/abs/2205.05638>.

Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., and Chen, M.-H. DoRA: Weight-decomposed low-rank adaptation. *arXiv preprint*, arXiv:2402.09353, 2024. URL <https://arxiv.org/abs/2402.09353>.

Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S., and Bossan, B. PEFT: State-of-the-art parameter-efficient fine-tuning. <https://github.com/huggingface/peft>, 2022.

Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Cao, H., Cheng, X., Chung, M., Grella, M., GV, K. K., He, X., Hou, H., Kazienko, P., Koccon, J., Kong, J., Koptyra, B., Lau, H., Mantri, K. S. I., Mom, F., Saito, A., Tang, X., Wang, B., Wind, J. S., Wozniak, S., Zhang, R., Zhang, Z., Zhao, Q., Zhou, P., Zhu, J., and Zhu, R.-J. RWKV: Reinventing RNNs for the transformer era. *arXiv preprint*, arXiv:2305.13048, 2023. URL <https://arxiv.org/abs/2305.13048>.

Sinha, K., Pineau, J., Forde, J., Ke, R. N., and Larochelle, H. ML reproducibility challenge 2020. *arXiv preprint*, arXiv:2104.08691, 2021. URL <https://arxiv.org/abs/2104.08691>.

Socher, R., Perelygin, A., Wu, J., Chuang, J., Manning, C. D., Ng, A. Y., and Potts, C. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of EMNLP 2013*, 2013. URL <https://aclanthology.org/D13-1170>.

Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., and Wei, F. Retentive network: A successor to transformer for large language models. *arXiv preprint*, arXiv:2307.08621, 2023. URL <https://arxiv.org/abs/2307.08621>.

Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., and Bowman, S. R. GLUE: A multi-task benchmark

and analysis platform for natural language understanding. In *Proceedings of EMNLP 2018 Black-boxNLP Workshop*, 2018. URL <https://arxiv.org/abs/1804.07461>.

Williams, A., Nangia, N., and Bowman, S. R. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of NAACL-HLT 2018*, 2018. URL <https://arxiv.org/abs/1704.05426>.

Zhang, Q., Chen, M., Bukharin, A., Karampatzakis, N., He, P., Cheng, Y., Chen, W., and Zhao, T. AdaLoRA: Adaptive budget allocation for parameter-efficient fine-tuning. *arXiv preprint*, arXiv:2303.10512, 2023. URL <https://arxiv.org/abs/2303.10512>.